

Lab Practicum 01: Empirical Parallel Computing

STUDENT NAME: Seiitkhan Zhannur

STUDENT ID: 230109009

HOST CPU / SOC MODEL: 12th Gen Intel Core i5-12450H

OPERATING SYSTEM & KERNEL: Microsoft Windows 11 Pro, 64-bit Windows NT
10.0.26200 (Build 26200)

TASK 1: Host Silicon Audit & Flynn's Taxonomy Mapping

CPU Model & Architecture: 12th Gen Intel(R) Core(TM) i5-12450H (2.00 GHz)

Base & Max Boost Clock : 2.00 GHz

Physical Hardware Cores: 8

Logical Cores (SMT/Threads): 12

L1 Data / Instruction Cache: 704 kb

L2 Cache (Per Core / Total): 7.0 mb

L3 Shared Last-Level Cache: 12.0 mb

SIMD Vector Extensions: AVX2

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

My CPU can work in different execution modes. SISD: A single CPU thread executes one sequence of instructions on one set of data. For example, a simple single-threaded program.

SIMD: One instruction can process multiple data values at the same time using vector instructions such as AVX/AVX2. This is useful for matrix calculations. MIMD: Multiple CPU cores can execute different instructions on different data at the same time. For example, different programs and threads running on different CPU cores.

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall

Threads / Workers	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time (s)	Speedu p	Efficienc y
1	2.1039	1.4759	1.5379	1.7059	1.00×	100.00%
2	1.0604	1.1155	1.0943	1.0901	1.56×	78.25%
4	0.6831	0.7089	0.7849	0.7256	2.35×	58.77%
8	0.7398	0.6254	0.5959	0.6537	2.61×	32.62%
16	0.7897	0.7695	0.7170	0.7587	2.25×	14.05%
32	1.0612	0.9848	1.1548	1.0669	1.60×	5.00%

Workload Algorithm: Prime Number Calculation up to 1,000,000

Language & Runtime: Python 3 / multiprocessing

Q2.1: Scaling Saturation & Hardware Contention Analysis:

My CPU is 12th Gen Intel Core i5-12450H with 8 physical cores and 12 logical processors. The best performance was at 8 workers, with average time 0.6537 seconds and speedup 2.61x. After 8 workers, performance started to decrease. At 16 workers the time increased to 0.7587 seconds, and at 32 workers it increased to 1.0669 seconds. This happens because the number of workers becomes higher than available CPU cores and logical processors, so the OS has more scheduling overhead and the processes compete for CPU and cache resources. Because of this, adding more workers does not always improve performance.

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap

Run 1	Output:	4591706	Error:	5408294	Time:	73.90 ms
Run 2	Output:	3822889	Error:	6177111	Time:	3.30 ms
Run 3	Output:	10000000	Error:	0	Time:	1.19 ms
Run 4	Output:	10000000	Error:	0	Time:	1.24 ms
Run 5	Output:	10000000	Error:	0	Time:	1.45 ms
Run 6	Output:	10000000	Error:	0	Time:	1.38 ms
Run 7	Output:	10000000	Error:	0	Time:	1.29 ms
Run 8	Output:	10000000	Error:	0	Time:	1.50 ms
Run 9	Output:	10000000	Error:	0	Time:	1.04 ms
Run 10	Output:	10000000	Error:	0	Time:	1.50 ms

Q3.1: Read-Modify-Write (RMW) Window

The race condition happens because `counter++` is not one operation. The CPU first loads the counter value, adds 1, and then stores the new value. Two threads can load the same value at the same time. For example, if the counter is 100, Thread A and Thread B can both load 100, both calculate 101, and both store 101. The correct value should be 102, so one increment is lost. This happened in my first two runs, where the final outputs were much lower than 10,000,000.

Q3.2: The Synchronization Penalty

Unlocked = 8.78 ms

Locked = 586.21 ms

The locked version is much slower because only one thread can update the shared counter at a time. Other threads have to wait for the lock. This removes the race condition and gives the correct result of 10,000,000, but it adds synchronization and waiting overhead.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

1. Parallel Fraction Derivation

$$T1 = 1.7059 \text{ s}$$

$$T2 = 1.0901 \text{ s}$$

$$p = 2 \times (T1 - T2) / T1$$

$$p = 2 \times (1.7059 - 1.0901) / 1.7059$$

$$p = 0.722$$

2. Amdahl Theoretical Speedup Ceiling

$$\text{Sequential fraction } (1 - p) = 0.278$$

$$S_{\text{max}} = 1 / (1 - p)$$

$$S_{\text{max}} = 1 / 0.278$$

$$S_{\text{max}} = 3.60\times$$

$$\text{Max Theoretical Speedup} = 3.60\times$$

3. Amdahl 64-Core Projection

$$S_{64} = 1 / [(1 - p) + (p / 64)]$$

$$S_{64} = 1 / [0.278 + (0.722 / 64)]$$

$$S_{64} = 3.46\times$$

$$\text{Projected Speedup on 64 Cores} = 3.46\times$$

4. Gustafson Scaled Speedup

$$S_{\text{Gustafson}} = (1 - p) + (p \times 64)$$

$$S_{\text{Gustafson}} = 0.278 + (0.722 \times 64)$$

$$S_{\text{Gustafson}} = 46.48\times$$

$$\text{Projected Scaled Speedup} = 46.48\times$$

Q4.1: Strong vs. Weak Scaling Analysis:

Amdahl's law assumes that the problem size stays the same when we add more cores.

Because 27.8% of my workload is sequential, this part becomes a bottleneck, so the maximum speedup is only around 3.60x. Gustafson's law assumes that when we add more

cores, we also increase the problem size. Most of this larger workload can run in parallel, so 64 cores can be used much more effectively. That is why Gustafson gives a much higher speedup of about 46.48x.

